Case Resolution Assistant — Demo Script

Setting the Scene

(before you share your screen)

> "So let me show you something we built to reduce the number of support cases that actually need human attention. The idea is simple — before we log a case, we let the AI try to solve it first."

Open the Site

(navigate to https://sgpt-demo-dev-ed.develop.my.site.com/gptfysupport/s/)

> "This is our public-facing support portal, built on Salesforce Experience Cloud. No login needed — anyone can come here and get help."

> "The page is clean — just a form. First Name, Last Name, Email, and a question. That's it."

Fill in the Form

(type in the details — use something realistic like a GPTfy or Salesforce question)

> "So let's say I'm a customer. I'll put in my name, my email, and then I'll type in my question."

(type the question — something like: "How do I connect GPTfy to my Salesforce org?")

> "This question is the key — it goes straight to our AI as the search query. No fluff, no extra fields."

> "Now I'll hit **Find a Solution**."

The Loading Screen

(spinner appears)

> "The moment I clicked that button, two things happened. Salesforce created a Case — it's already in the system. And a Record-Triggered Flow fired off on an **async path**, because Salesforce doesn't allow external API callouts in a synchronous transaction. The async path is what lets GPTfy make the callout cleanly."

> "Once GPTfy gets the response from our GCP RAG system, it writes it back to `Case.Description`. Now — we could have used Platform Events to push the response to the UI the moment it lands, but that approach gets complicated fast for a public site with no login. So we kept it simple with **polling** — the LWC checks every 2.5 seconds until the answer is there. 30 second timeout, graceful fallback. Does the job."

The Recommendation Appears

(recommendation shows up)

> "And there it is."

> "The AI has found a relevant article and surfaced the answer right here on the page. The customer hasn't had to wait on hold, open a ticket, or send an email. They have their answer in seconds."

> "Now we give them a choice."

Path A — Issue Resolved

(click "Yes, resolved")

> "If they say yes — the case that was created in the background gets automatically **closed**. No agent ever needs to touch it. That's a deflected case — zero effort, zero cost."

(show the Issue Resolved screen)

> "They see a confirmation, and they can submit another question if they need to."

Path B — Still Need Help

(click reset, go back, submit again, then click "No, still need help")

> "But if the answer didn't help — they click No."

(click "No, still need help")

> "Now look at this. The case is **already there**. We're not creating it now — it was created the moment they hit Find a Solution. We're just showing them the case number."

> "The case is open, it's in Salesforce, and a support agent will pick it up. The customer gets their case number, their email is confirmed, and they know someone will reach out."

(click Done)

> "Done resets everything back to the start. Clean slate."

The Salesforce Side *(optional — if time allows)*

(switch to Salesforce Service Console → Cases)

> "And if we jump into Salesforce, you can see the cases coming through — origin is Web, the question is the subject, the AI recommendation is stored in the description. Everything is traceable."

Close

> "So the stack is: LWC on Experience Cloud, a three-method Apex controller, a Record-Triggered Flow on an async path, and GPTfy calling GCP RAG. The Apex is `without sharing` so guest users can create and read Cases. The AI response lives in `Case.Description` — no custom fields needed."

> "And if the business wants to change the prompt — they open the Flow, update one value, save. No code, no deployment. That's the whole point."

> "That's the demo."

Total speaking time: ~4 to 5 minutes